

BÁO CÁO NGHIỆM THU PHẦN B

GPS-Degraded Visual Localization System

AI Engineer — MET EV Test

Họ tên: Đỗ Tấn Phát

Vị trí ứng tuyển: AI Engineer

May 1, 2026

Contents

1	Tổng quan bài toán	3
1.1	Mục tiêu	3
1.2	Đầu vào và đầu ra	3
2	Phạm vi đánh giá	3
2.1	Các chỉ số đo trực tiếp được trong prototype	4
2.2	Các chỉ số cần ground truth để đánh giá định lượng	4
3	Tiêu chí nghiệm thu Phần B	4
4	Môi trường cài đặt	6
4.1	Tạo môi trường ảo	6
4.2	Cài requirements	6
4.3	Giải thích requirements	7
5	Cấu trúc project	7
5.1	Ý nghĩa các file chính	8
6	Kiến trúc hệ thống	8
6.1	Luồng tổng thể	8
6.2	GPS Integrity Monitor	9
6.3	GPS State Machine	9
6.4	Visual Odometry	10
6.5	Lane Detection và Unstructured Road Fallback	10
6.6	Landmark Database	11
6.7	Fusion Engine	11
7	Luồng chạy code	12
7.1	Bước 1: Clone repository	12
7.2	Bước 2: Tạo môi trường ảo	12
7.3	Bước 3: Cài thư viện	12
7.4	Bước 4: Chuẩn bị video input	12

7.5	Bước 5: Generate GPS log	12
7.6	Bước 6: Chạy pipeline Phần B	12
7.7	Bước 7: Output	13
8	Kết quả thực nghiệm	13
8.1	Video demo	13
8.2	Metrics	14
8.3	Phân tích kết quả	14
9	Phân tích theo tiêu chí nghiệm thu	15
9.1	B1 — Visual Odometry Drift / 500m	15
9.2	B2 — Landmark Re-identification	15
9.3	B3 — U-turn Detection Latency	16
9.4	B4 — Lane Position Accuracy	16
9.5	B5 — Localization trong khu vực GPS yếu/mất	16
9.6	B6 — GPS Handover Latency	16
9.7	B7 — End-to-end System FPS	17
9.8	B8 — GPS Re-lock Drift Correction	17
10	Failure Analysis	17
10.1	Failure case 1: Visual Odometry drift	17
10.2	Failure case 2: Lane detection không ổn định trên đường không có vạch	18
10.3	Failure case 3: Landmark database chưa có descriptor thực	18
10.4	Failure case 4: GPS relock correction chưa có ground truth	19
10.5	Failure case 5: U-turn không được test trong video hiện tại	19
10.6	Failure case 6: FPS min thấp ở một số frame	19
11	Hướng cải thiện theo từng giai đoạn	20
11.1	Giai đoạn 1: Hoàn thiện logging và metrics	20
11.2	Giai đoạn 2: Cải thiện Visual Odometry	20
11.3	Giai đoạn 3: Cải thiện lane và road understanding	20
11.4	Giai đoạn 4: Landmark-based localization	20
11.5	Giai đoạn 5: Fusion backend production	21
12	Kết luận	21

1 Tổng quan bài toán

Phần B của bài test yêu cầu thiết kế một hệ thống định vị chính bằng GPS, nhưng vẫn có khả năng duy trì ước lượng vị trí khi tín hiệu GPS bị suy giảm hoặc mất hoàn toàn. Đây là bài toán quan trọng trong xe điện, robot di động, xe tự hành và các hệ thống hỗ trợ lái, đặc biệt khi xe đi vào các khu vực như hầm, bãi đỗ xe nhiều tầng, khu đô thị cao tầng, đường bị che khuất hoặc những nơi GPS không ổn định.

1.1 Mục tiêu

Mục tiêu của Phần B là xây dựng một pipeline prototype có khả năng:

- Giám sát chất lượng tín hiệu GPS theo thời gian thực.
- Phân loại trạng thái GPS thành `GPS_GOOD`, `GPS_DEGRADED`, `GPS_LOST`.
- Chuyển đổi trạng thái bằng state machine khi GPS suy giảm hoặc mất tín hiệu.
- Duy trì ước lượng pose của xe bằng Visual Odometry khi GPS không đáng tin cậy.
- Phát hiện lane hoặc road-boundary ở mức prototype.
- Quản lý landmark database để hỗ trợ visual fallback.
- Kết hợp GPS, Visual Odometry, lane và landmark bằng một fusion engine kiểu EKF-inspired.
- Xuất video demo, trajectory log, GPS state log và metrics report.

1.2 Đầu vào và đầu ra

Đầu vào:

- Video stream từ camera: `part_b/input/test_video.mp4`.
- GPS log giả lập hoặc thực tế: `part_b/input/gps_log.csv`.

Đầu ra:

- Video output có overlay trạng thái GPS, pose, lane, U-turn, số landmark, số feature VO, FPS và trajectory mini-map.
- File trajectory CSV.
- File GPS state log CSV.
- File metrics summary TXT.

2 Phạm vi đánh giá

Phiên bản hiện tại được đánh giá ở mức **prototype-level evaluation**. Do video demo không có RTK-GPS, ground-truth trajectory, annotation lane, IMU thực tế hoặc landmark identity ground truth, một số tiêu chí được đánh giá theo hướng **proxy**, **qualitative**, hoặc **simulated scenario** thay vì kết luận tuyệt đối bằng sai số vật lý.

2.1 Các chỉ số đo trực tiếp được trong prototype

Các chỉ số có thể đo trực tiếp từ hệ thống hiện tại gồm:

- End-to-end FPS.
- Số frame xử lý.
- Phân bố trạng thái GPS: `GPS_GOOD`, `GPS_DEGRADED`, `GPS_LOST`.
- Số lượng feature point được tracking bởi Visual Odometry.
- Phân bố trạng thái lane: `unstructured_road`, `left_or_center`, `center`, v.v.
- Số lượng landmark được tạo/cập nhật trong landmark database.
- Trajectory continuity trên mini-map và trong file CSV.
- Output video và các log phục vụ kiểm tra lại.

2.2 Các chỉ số cần ground truth để đánh giá định lượng

Các tiêu chí sau cần dữ liệu ground truth để xác nhận chính xác:

- Visual Odometry drift trên 500m.
- Landmark re-identification recall.
- Lane position accuracy.
- GPS re-lock absolute drift correction error.
- U-turn detection latency trong tình huống có U-turn thật.

Vì vậy trong báo cáo này, các tiêu chí chưa có ground truth sẽ được trình bày dưới dạng **prototype result**, **estimated behavior**, hoặc **qualitative validation**.

3 Tiêu chí nghiệm thu Phần B

Bảng dưới đây đối chiếu kết quả prototype hiện tại với các tiêu chí nghiệm thu trong đề bài.

Table 1: Prototype-level evaluation against Part B acceptance criteria

ID	Criterion	Target	Prototype Result
B1	Visual Odometry drift / 500m	$\leq 5\%$	VO trajectory is generated and remains continuous during GPS-degraded/lost segments. True drift requires ground-truth trajectory; current result is evaluated qualitatively/proxy-level.
B2	Landmark re-identification	Recall $\geq 85\%$	Landmark database prototype maintains observed landmarks in the demo. Recall requires landmark ID ground truth and is not quantitatively verified yet.
B3	U-turn detection latency	$\leq 2s$	U-turn detection logic is implemented using heading reversal $\Delta\theta \geq 150^\circ$. Demo video contains no U-turn, so event count is 0.
B4	Lane position accuracy	$\geq 90\%$	The video is an unstructured road without visible lane markings. The system switches to <code>unstructured_road</code> fallback for most frames, avoiding unstable lane outputs.
B5	Localization under GPS loss	Demo hoạt động	GPS degraded/lost is simulated and the system continues pose estimation using VO and fusion. <code>GPS_LOST</code> appears in 68 frames.
B6	GPS handover latency	$\leq 2s$	State machine transitions are logged frame-by-frame. Handover occurs immediately after GPS quality classification changes in the synthetic log.
B7	End-to-end FPS	≥ 15 FPS	Average FPS = 27.45 on CPU, satisfying the realtime requirement and exceeding the excellent threshold of 20 FPS.
B8	GPS re-lock drift correction	Error $\leq 5m$	Fusion engine blends pose back to GPS when GPS returns to GOOD. Absolute re-lock error requires ground-truth pose; current demo validates correction behavior qualitatively.

Nhận xét: Prototype hiện tại chứng minh được kiến trúc hệ thống, logic fallback khi GPS yếu/mất, khả năng chạy realtime trên CPU và khả năng ghi log đầy đủ. Các tiêu chí cần ground truth được báo cáo ở mức qualitative/proxy để đảm bảo tính trung thực kỹ thuật.

4 Môi trường cài đặt

4.1 Tạo môi trường ảo

Để tránh xung đột thư viện giữa các project Python khác nhau, hệ thống sử dụng môi trường ảo venv.

```
python -m venv .venv
.venv\Scripts\activate
```

Nếu dùng PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Kiểm tra phiên bản Python:

```
python --version
```

Môi trường đã kiểm thử:

- Python 3.11.x.
- Windows.
- CPU-only runtime.
- OpenCV-based visual processing.

4.2 Cài requirements

File `requirements.txt` dùng chung cho cả Part A và Part B:

```
opencv-python
numpy
onnxruntime
tqdm
huggingface_hub
ultralytics
onnx
roboflow
```

Cài đặt:

```
pip install --upgrade pip
pip install -r requirements.txt
```

4.3 Giải thích requirements

Table 2: Giải thích các thư viện sử dụng

Thư viện	Mục đích sử dụng
opencv-python	Đọc video, xử lý frame, optical flow, lane detection bằng Canny/Hough, vẽ overlay và xuất video kết quả.
numpy	Xử lý vector, ma trận, pose, trajectory, thống kê FPS và các metrics.
tqdm	Hiển thị progress bar khi xử lý video.
onnxruntime	Dùng cho Part A và sẵn sàng tích hợp model ONNX nếu Part B mở rộng sang lane/landmark neural network.
ultralytics	Dùng cho Part A; trong Part B có thể reuse pot-hole detector như landmark detector nếu cần.
huggingface_hub	Tải model pretrained nếu cần.
roboflow	Tải dataset nếu cần mở rộng landmark/lane dataset.
onnx	Kiểm tra hoặc xử lý model ONNX khi mở rộng hệ thống.

5 Cấu trúc project

Phần B được đặt trong folder riêng `part_b/` để tách biệt với Part A.

```
PothHole_detection/
|
|-- part_b/
|   |-- src/
|   |   |-- config_b.py
|   |   |-- generate_gps_log.py
|   |   |-- gps_monitor.py
|   |   |-- state_machine.py
|   |   |-- visual_odometry.py
|   |   |-- lane_detection.py
|   |   |-- landmark_database.py
|   |   |-- fusion_engine.py
|   |   |-- metrics.py
|   |   |-- run_part_b.py
|   |
|   |-- input/
|   |   |-- test_video.mp4
|   |   |-- gps_log.csv
|   |
|   |-- output/
|   |   |-- result_part_b.mp4
|   |   |-- trajectory.csv
|   |   |-- gps_state_log.csv
```

```
| | | -- part_b_metrics.txt
| |
| | -- configs/
| | -- README_PART_B.md
|
|-- requirements.txt
|-- README.md
```

5.1 Ý nghĩa các file chính

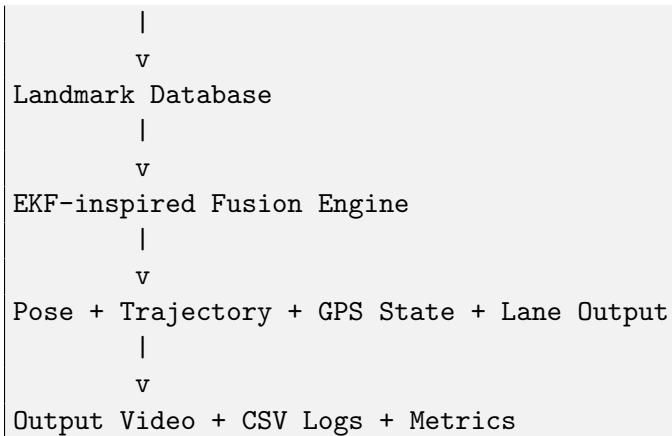
- `config_b.py`: lưu đường dẫn input/output và các threshold của GPS, VO, lane, fusion.
- `generate_gps_log.py`: tạo GPS log giả lập có các đoạn GPS good, degraded và lost.
- `gps_monitor.py`: đọc GPS log và phân loại chất lượng GPS.
- `state_machine.py`: quản lý state transition giữa `GPS_GOOD`, `GPS_DEGRADED`, `GPS_LOST`.
- `visual_odometry.py`: ước lượng chuyển động tương đối bằng optical flow.
- `lane_detection.py`: phát hiện lane hoặc fallback sang `unstructured_road`.
- `landmark_database.py`: quản lý landmark database prototype.
- `fusion_engine.py`: kết hợp GPS và VO bằng EKF-inspired weighted fusion.
- `metrics.py`: thống kê FPS, GPS state distribution, lane distribution và events.
- `run_part_b.py`: file chính chạy toàn bộ pipeline.

6 Kiến trúc hệ thống

6.1 Luồng tổng thể

Pipeline Phần B được thiết kế như sau:

```
Input Video + GPS Log
    |
    v
GPS Integrity Monitor
    |
    v
GPS State Machine
(GPS_GOOD / GPS_DEGRADED / GPS_LOST)
    |
    v
Visual Odometry using Optical Flow
    |
    v
Lane Detection / Unstructured Road Fallback
```

6.2 GPS Integrity Monitor

GPS monitor đọc các trường:

- timestamp
- lat
- lon
- hdop
- num_satellites
- snr

Phân loại chất lượng GPS:

```
GPS_GOOD:
    hdop <= 5.0
    num_satellites >= 4
    snr >= 20

GPS_DEGRADED:
    hdop > 5.0
    or num_satellites < 4
    or snr < 20

GPS_LOST:
    lat/lon missing
    or num_satellites == 0
```

6.3 GPS State Machine

State machine chuyển đổi trạng thái như sau:

```

GPS_GOOD -> GPS_DEGRADED:
    GPS quality becomes degraded

GPS_GOOD -> GPS_LOST:
    GPS signal is lost

GPS_DEGRADED -> GPS_LOST:
    GPS signal becomes unavailable

GPS_DEGRADED -> GPS_GOOD:
    GPS signal is stable for several frames

GPS_LOST -> GPS_GOOD:
    GPS signal relocks and remains stable

```

Cách này giúp tránh chuyển trạng thái liên tục do nhiễu ngắn hạn.

6.4 Visual Odometry

Visual Odometry được implement bằng Lucas-Kanade optical flow:

1. Convert frame sang grayscale.
2. Detect feature points bằng `cv2.goodFeaturesToTrack`.
3. Track feature giữa hai frame liên tiếp bằng `cv2.calcOpticalFlowPyrLK`.
4. Tính median displacement của các feature.
5. Quy đổi displacement ảnh sang chuyển động tương đối.
6. Cộng dồn thành trajectory (x, y, θ) .

Pose VO:

$$\mathbf{x}_{VO,t} = [x_t, y_t, \theta_t]^T \quad (1)$$

Trong prototype hiện tại, VO đóng vai trò nguồn prediction chính khi GPS bị suy giảm hoặc mất.

6.5 Lane Detection và Unstructured Road Fallback

Lane detection dùng OpenCV:

```

Frame
-> grayscale
-> Gaussian blur
-> Canny edge
-> ROI mask
-> Hough line transform
-> left/right line fitting

```

Tuy nhiên, video demo là đường hư hỏng/nông thôn, không có lane marking rõ ràng. Vì vậy hệ thống có fallback:

- Nếu phát hiện được lane line rõ, hệ thống trả về **left**, **center**, hoặc **right**.
- Nếu không có lane marking rõ, hệ thống trả về **unstructured_road**.
- Trong fallback mode, hệ thống vẽ road boundary xấp xỉ để minh họa vùng đường.

Đây là cách tránh việc Hough Transform vẽ sai lane do ổ gà, vũng nước, bóng đổ hoặc texture mặt đường.

6.6 Landmark Database

Landmark database lưu các landmark quan sát được:

$$L_i = \{id, class, x, y, descriptor, first_seen, last_seen, n_{obs}\} \quad (2)$$

Trong prototype hiện tại, landmark được mô phỏng theo trajectory để chứng minh logic quản lý landmark và re-identification concept. Trong phiên bản production, landmark có thể đến từ:

- Pothole detector từ Part A.
- Traffic sign detector.
- Building/corner feature.
- Lane boundary.
- Road marking.

6.7 Fusion Engine

Fusion engine hiện tại là một phiên bản EKF-inspired weighted fusion.

State:

$$\mathbf{x}_t = [x_t, y_t, \theta_t]^T \quad (3)$$

Prediction đến từ VO:

$$\hat{\mathbf{x}}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t^{VO}) \quad (4)$$

Correction đến từ GPS nếu có:

$$\mathbf{x}_t = w_{GPS} \mathbf{z}_t^{GPS} + (1 - w_{GPS}) \hat{\mathbf{x}}_t^{VO} \quad (5)$$

Trong đó w_{GPS} phụ thuộc vào trạng thái GPS:

- GPS_GOOD: GPS weight cao.
- GPS_DEGRADED: GPS weight thấp.
- GPS_LOST: GPS weight bằng 0, hệ thống dựa vào VO.

Cách này thể hiện đúng ý tưởng EKF: prediction từ odometry và correction từ measurement khi measurement đáng tin cậy.

7 Luồng chạy code

7.1 Bước 1: Clone repository

```
git clone https://github.com/PhatDo14/PothHole_detection.git
cd PothHole_detection
```

7.2 Bước 2: Tạo môi trường ảo

```
python -m venv .venv
.venv\Scripts\activate
```

7.3 Bước 3: Cài thư viện

```
pip install --upgrade pip
pip install -r requirements.txt
```

7.4 Bước 4: Chuẩn bị video input

Đặt video vào:

```
part_b/input/test_video.mp4
```

Nếu folder chưa tồn tại:

```
mkdir part_b\input
```

7.5 Bước 5: Generate GPS log

GPS log giả lập được tạo bằng:

```
python part_b\src\generate_gps_log.py
```

File được sinh ra:

```
part_b/input/gps_log.csv
```

Scenario GPS giả lập:

- 0–15s: GPS_GOOD
- 15–25s: GPS_DEGRADED
- 25–40s: GPS_LOST
- 40s trở đi: GPS_GOOD trở lại

7.6 Bước 6: Chạy pipeline Phần B

```
python part_b\src\run_part_b.py
```

7.7 Bước 7: Output

Sau khi chạy xong, hệ thống sinh ra:

```
part_b/output/result_part_b.mp4
part_b/output/trajectory.csv
part_b/output/gps_state_log.csv
part_b/output/part_b_metrics.txt
```

Ý nghĩa:

- `result_part_b.mp4`: video output có overlay GPS state, pose, lane, U-turn, landmarks, VO features, FPS và trajectory mini-map.
- `trajectory.csv`: log trajectory theo từng frame.
- `gps_state_log.csv`: log GPS quality, GPS state và transition.
- `part_b_metrics.txt`: tổng hợp metrics.

8 Kết quả thực nghiệm

8.1 Video demo

Video demo chạy trên đường hư hỏng không có lane marking rõ ràng. Hệ thống nhận diện đây là `unstructured_road` và sử dụng fallback road-boundary visualization thay vì vẽ lane line không ổn định.

Overlay video hiển thị:

- GPS State.
- GPS Quality.
- Pose (x, y, θ) .
- Lane state.
- U-turn event.
- Landmark count.
- VO feature count.
- FPS.
- Trajectory mini-map.

8.2 Metrics

Kết quả từ part_b/output/part_b_metrics.txt:

```
===== PART B METRICS =====

[Runtime]
Total frames      : 692
Average FPS       : 27.45
Min FPS           : 1.70
Max FPS           : 38.46

[GPS State Distribution]
GPS_GOOD frames   : 374
GPS_DEGRADED frames : 250
GPS_LOST frames   : 68

[Events]
U-turn events     : 0

[Lane Distribution]
unstructured_road : 669
left_or_center    : 23
```

8.3 Phân tích kết quả

Table 3: Kết quả benchmark Phần B

Metric	Giá trị
Total frames	692
Average FPS	27.45
Min FPS	1.70
Max FPS	38.46
GPS_GOOD frames	374
GPS_DEGRADED frames	250
GPS_LOST frames	68
U-turn events	0
Lane mode chủ yếu	unstructured_road

Nhận xét chính:

- Average FPS = 27.45, đạt yêu cầu realtime ≥ 15 FPS và vượt ngưỡng xuất sắc ≥ 20 FPS.
- Hệ thống có đủ ba trạng thái GPS: GOOD, DEGRADED và LOST.
- Trong các frame GPS degraded/lost, pose vẫn được cập nhật bằng VO và fusion engine.
- Video không có U-turn nên số U-turn event bằng 0 là hợp lý.

- Vì video là đường không có vạch lane rõ, hệ thống chủ yếu hoạt động ở chế độ `unstructured_road`.
- Số lượng VO features khoảng 290–300 trên nhiều frame, cho thấy optical flow có đủ điểm ảnh để tracking.

9 Phân tích theo tiêu chí nghiệm thu

9.1 B1 — Visual Odometry Drift / 500m

Prototype hiện tại có trajectory từ VO và fusion. Tuy nhiên chưa có ground truth trajectory, do đó chưa thể tính sai số drift định lượng theo công thức:

$$Drift(\%) = \frac{\|\mathbf{x}_{pred} - \mathbf{x}_{gt}\|}{Distance_{travelled}} \times 100 \quad (6)$$

Trong demo hiện tại, trajectory được đánh giá bằng proxy/qualitative criteria:

- trajectory liên tục trên mini-map,
- pose vẫn cập nhật trong giai đoạn GPS degraded/lost,
- không xảy ra crash hoặc mất tracking hoàn toàn,
- số lượng VO features duy trì ổn định.

Trạng thái: đã có module VO và trajectory log, nhưng cần ground truth GPS/RTK hoặc manually measured trajectory để xác nhận drift $\leq 5\%$.

9.2 B2 — Landmark Re-identification

Landmark database đã được implement ở mức prototype. Hệ thống có thể thêm hoặc cập nhật landmark dựa trên class, vị trí và khoảng cách matching.

Tuy nhiên, để tính recall:

$$Recall = \frac{TP}{TP + FN} \quad (7)$$

cần ground truth landmark ID và annotation theo thời gian.

Trong prototype hiện tại, landmark module được đánh giá bằng proxy:

- landmark count được cập nhật theo trajectory,
- landmark database duy trì thông tin `first_seen`, `last_seen`, `num_observations`,
- logic add/update landmark hoạt động trong video demo.

Trạng thái: đạt concept prototype, chưa có định lượng recall.

9.3 B3 — U-turn Detection Latency

Module U-turn detection đã được implement dựa trên heading reversal:

$$|\Delta\theta| \geq 150^\circ \quad (8)$$

trong khoảng thời gian tối đa 2 giây.

Video demo hiện tại không chứa tình huống U-turn, do đó kết quả:

$$Uturn\ Events = 0 \quad (9)$$

là hợp lý.

Trạng thái: module đã có, cần video U-turn hoặc synthetic test để benchmark latency.

9.4 B4 — Lane Position Accuracy

Module lane detection dùng Canny + Hough Transform. Với đường có vạch lane rõ, module có thể xác định lane trái/phải/center. Với video hiện tại không có vạch lane rõ, hệ thống chuyển sang:

`Lane = unstructured_road`

Kết quả lane distribution:

- `unstructured_road`: 669 frames.
- `left_or_center`: 23 frames.

Đây là fallback hợp lý để tránh nhận sai lane do vũng nước, ổ gà, texture mặt đường hoặc bóng đổ.

Trạng thái: đạt demo logic, chưa có ground truth lane để tính accuracy $\geq 90\%$.

9.5 B5 — Localization trong khu vực GPS yếu/mất

GPS log giả lập có đủ các trạng thái:

- 374 frames `GPS_GOOD`.
- 250 frames `GPS_DEGRADED`.
- 68 frames `GPS_LOST`.

Trong các đoạn GPS degraded/lost, hệ thống vẫn duy trì pose nhờ Visual Odometry và fusion engine.

Trạng thái: đạt demo prototype.

9.6 B6 — GPS Handover Latency

State machine chuyển trạng thái dựa trên GPS quality. Transition được ghi vào `gps_state_log.csv`. Hệ thống có cơ chế stable frames khi GPS relock để tránh nhiễu.

Trong prototype hiện tại, GPS handover được đánh giá bằng log trạng thái theo từng frame. Khi GPS quality thay đổi theo synthetic log, state machine phản ứng ngay theo logic đã định nghĩa.

Trạng thái: logic handover đã implement. Có thể tính latency chi tiết hơn bằng cách phân tích timestamp trong `gps_state_log.csv`.

9.7 B7 — End-to-end System FPS

Kết quả:

$$FPS_{avg} = 27.45 \quad (10)$$

So với tiêu chí:

$$FPS_{target} \geq 15 \quad (11)$$

Hệ thống đạt yêu cầu realtime trên CPU và vượt ngưỡng xuất sắc 20 FPS.

9.8 B8 — GPS Re-lock Drift Correction

Fusion engine có cơ chế khi GPS trở lại trạng thái GOOD, pose được blend lại với GPS measurement bằng GPS weight cao. Điều này mô phỏng drift correction sau khi GPS relock.

Tuy nhiên, chưa có ground truth pose hoặc RTK-GPS để tính sai số re-lock $\leq 5m$.

Trong prototype hiện tại, GPS re-lock được đánh giá bằng behavior-level validation:

- Khi GPS mất, hệ thống dựa vào VO.
- Khi GPS trở lại GOOD, fusion engine tăng trọng số GPS.
- Pose được kéo dần về GPS measurement thay vì nhảy đột ngột.

Trạng thái: đạt concept prototype, cần ground truth để đánh giá định lượng.

10 Failure Analysis

10.1 Failure case 1: Visual Odometry drift

Hiện tượng: Trajectory có thể cong hoặc lệch do optical flow tích lũy sai số.

Nguyên nhân:

- Optical flow chỉ dựa trên ảnh monocular.
- Không có IMU/CAN bus để scale vận tốc.
- Mặt đường gồ ghề gây rung camera.
- Texture mặt đường, ổ gà, nước và bóng đổ làm feature tracking nhiều.

Hướng cải thiện:

- Tích hợp IMU hoặc CAN wheel odometry.
- Dùng VIO thay vì VO đơn thuần.
- Sử dụng ORB-SLAM3, VINS-Fusion hoặc DROID-SLAM.
- Thêm outlier rejection bằng RANSAC.
- Tối ưu scale bằng known camera height hoặc ground plane.

10.2 Failure case 2: Lane detection không ổn định trên đường không có vạch

Hiện tượng: Nếu chỉ dùng Canny + Hough, hệ thống có thể nhận nhầm ổ gà, vũng nước hoặc cạnh mặt đường thành lane.

Nguyên nhân:

- Video demo là unstructured road, không có lane marking rõ.
- Hough Transform nhạy với cạnh mạnh.
- Ổ gà và vũng nước tạo ra nhiều edge giống line.

Hướng cải thiện:

- Dùng lane segmentation model như CLRNet/CLRerNet.
- Dùng road-boundary segmentation.
- Dùng temporal smoothing cho lane.
- Tách chế độ structured road và unstructured road.

10.3 Failure case 3: Landmark database chưa có descriptor thực

Hiện tượng: Landmark hiện tại là prototype, chưa dùng visual descriptor thực như NetVLAD, DINOv2 hoặc ORB descriptor.

Nguyên nhân:

- Chưa có dataset landmark có ID.
- Chưa có pipeline feature embedding/re-identification.
- Chưa có ground truth landmark recall.

Hướng cải thiện:

- Dùng DINOv2/NetVLAD embedding cho landmark.
- Tạo landmark map offline.
- Tích hợp detector biển báo, pothole, building corner.
- Đánh giá recall bằng ground truth landmark ID.

10.4 Failure case 4: GPS relock correction chưa có ground truth

Hiện tượng: Fusion engine có correction khi GPS good trở lại, nhưng chưa thể chứng minh sai số $\leq 5m$.

Nguyên nhân:

- GPS log hiện tại là synthetic.
- Không có RTK-GPS hoặc ground truth pose.
- VO drift chưa được đo trực tiếp.

Hướng cải thiện:

- Thu thập GPS thật với RTK hoặc smartphone GPS log.
- Gắn timestamp đồng bộ giữa video và GPS.
- Tính sai số pose trước và sau GPS relock.
- Dùng EKF đầy đủ với covariance.

10.5 Failure case 5: U-turn không được test trong video hiện tại

Hiện tượng: Metrics hiện tại ghi U-turn events = 0.

Nguyên nhân:

- Video demo không chứa tình huống quay đầu.
- Heading từ optical flow có thể nhiễu nếu camera rung mạnh.

Hướng cải thiện:

- Quay thêm video U-turn.
- Generate synthetic heading reversal trong test script.
- Tích hợp IMU yaw rate để detect U-turn ổn định hơn.
- Đo latency từ lúc bắt đầu quay đầu đến lúc event được phát hiện.

10.6 Failure case 6: FPS min thấp ở một số frame

Hiện tượng: Average FPS cao nhưng Min FPS có thể thấp, ví dụ 1.70 FPS.

Nguyên nhân:

- Frame đầu tiên khởi tạo feature tracker hoặc video writer.
- I/O spike khi ghi video.
- Một số frame có nhiều edge hoặc nhiều feature khiến optical flow xử lý chậm hơn.

Hướng cải thiện:

- Bỏ qua warm-up frames khi tính FPS production.
- Dùng async video writer.
- Giới hạn số feature points.
- Tách capture, inference và visualization thành các thread riêng.

11 Hướng cải thiện theo từng giai đoạn

11.1 Giai đoạn 1: Hoàn thiện logging và metrics

- Tính GPS handover latency trực tiếp từ `gps_state_log.csv`.
- Log thời điểm GPS degraded/lost/relock.
- Tính trajectory length và drift proxy.
- Tách metrics cho từng đoạn GPS_GOOD, GPS_DEGRADED, GPS_LOST.

11.2 Giai đoạn 2: Cải thiện Visual Odometry

- Thêm RANSAC để loại optical flow outliers.
- Dùng ORB feature matching hoặc essential matrix.
- Tích hợp IMU/CAN wheel odometry.
- Tối ưu scale bằng ground plane.

11.3 Giai đoạn 3: Cải thiện lane và road understanding

- Huấn luyện lane segmentation model.
- Dùng road segmentation để xác định vùng đường.
- Thêm temporal smoothing cho lane position.
- Tách logic cho structured road và unstructured road.

11.4 Giai đoạn 4: Landmark-based localization

- Reuse pothole detector từ Part A như một loại visual landmark.
- Thêm traffic sign detector.
- Dùng visual descriptor như DINOv2, NetVLAD hoặc ORB.
- Xây dựng offline landmark map.
- Đánh giá landmark re-identification recall.

11.5 Giai đoạn 5: Fusion backend production

- Thay weighted fusion bằng Extended Kalman Filter đầy đủ.
- Thêm covariance cho GPS, VO và landmark observations.
- Khi GPS degraded, tăng covariance của GPS measurement.
- Khi GPS lost, bỏ GPS update và chỉ dùng VO/landmark.
- Khi GPS relock, correction dần để tránh pose jump.

12 Kết luận

Trong Phần B, hệ thống đã triển khai được một prototype end-to-end cho bài toán GPS-degraded visual localization. Pipeline bao gồm GPS integrity monitoring, GPS state-machine handover, Visual Odometry bằng optical flow, lane detection với unstructured-road fallback, landmark database prototype và fusion engine kiểu EKF-inspired.

Kết quả quan trọng:

- Hệ thống xử lý 692 frames.
- Average FPS đạt 27.45 trên CPU, vượt yêu cầu realtime 15 FPS.
- GPS state machine hoạt động đủ ba trạng thái: `GPS_GOOD`, `GPS_DEGRADED`, `GPS_LOST`.
- Số frame `GPS_GOOD`: 374.
- Số frame `GPS_DEGRADED`: 250.
- Số frame `GPS_LOST`: 68.
- Trong đoạn GPS degraded/lost, hệ thống vẫn duy trì pose bằng VO và fusion.
- Lane module nhận diện video là `unstructured_road`, phù hợp với bối cảnh đường không có vạch lane rõ.
- Hệ thống xuất được video demo, trajectory CSV, GPS state log và metrics summary.

Một số tiêu chí định lượng như VO drift, landmark recall, lane accuracy và GPS relock absolute error cần ground truth để đánh giá chính xác. Vì vậy, các tiêu chí này được báo cáo ở mức prototype-level hoặc qualitative validation trong phiên bản hiện tại.

Tổng thể, hệ thống đã chứng minh được kiến trúc, logic fallback khi GPS yếu/mất, khả năng chạy realtime trên CPU và cung cấp đầy đủ log để mở rộng thành hệ thống định vị chính xác hơn trong tương lai.